

Wakeel — Sovereign Regulatory Agent Factory

PRD v3.0 (*supersedes v2.0*) **Target v1.0 ship: June 7, 2026** **Developer delivery milestone: May 30, 2026**

Changes from v2.0:

- Arabic intake support added to in-scope (Interviewer agent)
 - Streamlit chat UI on port 8001 promoted from optional to in-scope
 - LLM client implementation requirements expanded (see SOW for full detail)
 - Corpus scope expanded from 2 to 4 UAE Federal statutes (Labour Law, PDPL, Commercial Transactions Law key chapters, Civil Transactions Law contract articles)
-

1. One-liner

Wakeel turns your organization's interpretation of UAE regulation into deployable AI copilots. Describe the workflow in English or Arabic, debate the requirements with our agent council, and ship a specialized compliance assistant tuned to your org's stance — without writing code.

2. Product thesis

Law isn't deterministic. Compliance is interpretation.

Public statutes are public. Anyone can read PDPL Article 7, UAE Labour Law Article 43, or the Commercial Companies Law. But how a bank reads those articles is different from how a hospital reads them, and different again from how a fintech startup reads them. That difference lives in internal memos, partner meetings, historical decisions, and the heads of senior counsel. It's tacit, it doesn't scale, and it doesn't survive turnover.

Wakeel encodes that institutional interpretation layer as deployable agents. The agent factory interviews the team, debates the stance, builds a configured copilot, and validates it works — all in one session. The output is an assistant that reads documents the way *your organization* reads them.

The moat: public laws are public. Interpretations are private. Wakeel owns the layer

between them.

3. Why this product works

Stickiness. Every override, edit, and decision the copilot logs teaches the org's private interpretation layer. Leaving Wakeel means losing that learned knowledge.

Expansion path. Start with NDA review. Add employment contracts. Add data-processing agreements. Add PDPL audits. Each new copilot ships against the same org corpus and learns from the same override history.

Defensibility. Competitors can scrape statutes. They cannot scrape your internal interpretations, your risk thresholds, or your history of which clauses your counsel will and won't accept.

Sovereignty. All processing happens on UAE-resident infrastructure. The interpretation layer never leaves the tenant. This is an architectural property, not a marketing claim.

4. Target users

Primary buyer: General Counsel or Head of Compliance at a UAE-domiciled mid-market enterprise in a regulated sector (financial services, healthcare, insurance, real estate, telco).

Primary user: In-house counsel, compliance officers, and contract reviewers in the same org.

Demo persona: Sarah, in-house counsel at a Dubai-based fintech. She reviews 30+ vendor NDAs per quarter. Her firm's stance on data-handling clauses is more conservative than the public PDPL baseline. Today she does this manually. With Wakeel, she mints an "NDA reviewer tuned to our fintech stance" once, then uses it daily.

5. v1.0 scope

In scope

- Single-tenant agent factory: one workflow, one copilot per session
- Two operating modes: **build** (factory mints a copilot) and **use** (copilot reviews a document)

- One template family in v1: **document review copilots** (NDA in v1; employment, services as stretch)
- Org adaptation layer via JSON config: custom risk thresholds, interpretation overrides, optional internal-policy corpus add-on
- Citation verification on every regulatory claim
- Counter-proposal drafting for flagged clauses
- Single API endpoint serving both modes
- **Arabic input support in the Interviewer agent (build mode)** — user can describe their workflow in Arabic
- **Streamlit chat-based UI on port 8001** — mode toggle, conversational interface for build mode, document submission for use mode, audit trail visible
- Audit trail logging every agent interaction

Stretch (only if Days 1–4 are clean)

- Bilingual document ingestion (Arabic NDAs in use mode)
- Second copilot template (employment contract review)

Out of scope (deferred to v1.1+)

- Arabic output (findings, counter-proposals in Arabic) — input only in v1
- Multi-document case files
- Free-zone law coverage (DIFC, ADGM)
- Web UI for org-adaptation config management (JSON file only in v1)
- Multi-tenant SaaS infrastructure
- Continuous learning from user overrides (logged in v1, not fed back)
- Litigation, court filings, e-discovery
- User accounts, authentication, billing

This list is the line. Nothing in “out of scope” gets built before June 7. If we slip on in-scope items, the cut sequence is: stretch items first, then chat UI fancy features (keep barebones chat), then drop counter-proposal critique loop.

6. The two operating modes

Build mode (`mode: "build"`)

A user describes a regulatory workflow in English or Arabic. The agent factory interviews them, debates the stance, configures a new copilot, validates it, and returns a `copilot_id` plus a config artifact. Typical run time: 60–120 seconds.

Use mode (`mode: "use"`)

The user submits a document plus an existing `copilot_id`. The configured copilot reviews the document against its tuned stance, verifies citations, drafts counter-proposals, and returns a structured review. Typical run time: 30–90 seconds.

7. Agent architecture

Five distinct feedback loops across two modes. Orchestration via **LangGraph** for explicit state, branching, and loops.

Build mode agents

Agent	Role	Tier	Notes
Interviewer	Extracts structured requirements from the user	Standard	Accepts Arabic or English input. Responds in the user’s language. Routes structured output to downstream agents in English.
Stance Debater A — Strict Compliance	Argues for conservative interpretations	Reasoning	English
Stance Debater B — Business Practicality	Argues for workable positions	Reasoning	English
Architect	Synthesizes requirements + debate into a copilot	Reasoning	English

	config		
Builder	Instantiates the copilot with prompts, schema, retrieval rules	Standard	English
Validator	Runs sample inputs through the new copilot, rejects if substandard	Standard	English

Use mode agents (the produced copilot)

Agent	Role	Tier
Reviewer	Reviews documents against the org-tuned stance	Reasoning
Citation Verifier	Validates citations against the corpus via exact text retrieval	Standard
Counter-Proposal Drafter	Drafts replacement clauses for flagged risks	Standard
Synthesis	Assembles final output with confidence scores and audit trail	Standard

The five feedback loops

Loop 1 — Stance debate (build mode): Debater A and Debater B exchange arguments across 2–3 rounds before the Architect synthesizes. Each round logged. Architect can request additional rounds.

Loop 2 — Validation rejection (build mode): Validator runs sample inputs through the new copilot. If outputs are substandard, Builder receives feedback and revises. Max 3 iterations.

Loop 3 — Requirements clarification (build mode): If the Architect detects ambiguity in requirements, it escalates back to the Interviewer for follow-up questions.

Loop 4 — Citation rejection (use mode): The killer feature. Reviewer’s findings are checked by Citation Verifier against the corpus. Hallucinated cites are rejected with the actual article text returned. Max 3 retries per finding.

Loop 5 — Counter-proposal critique (use mode): Drafter's replacement clauses are checked by Reviewer against the same statute that triggered the original flag. Bad drafts go back. Max 3 iterations.

Every loop iteration is logged to the audit trail with agent name, action, decision, and reason.

8. The org adaptation layer (the moat)

Each copilot instance is configured by an org-specific JSON document:

```
{
  "org_id": "fintech_co_001",
  "interpretation_overrides": [
    {
      "law": "Federal Decree-Law 45 of 2021",
      "article": "Article 7",
      "stance": "We require explicit consent re-confirmation for any data shared",
      "rationale": "Banking partner contractual requirement; conservative posture"
    }
  ],
  "risk_thresholds": {
    "high_flag_confidence_min": 0.80,
    "medium_flag_confidence_min": 0.60,
    "auto_escalate_to_human": ["data_transfer_abroad", "indemnity_caps"]
  },
  "internal_corpus_additions": [
    "/data/org/fintech_co_001/internal_policy_pdpl_v3.pdf"
  ],
  "output_preferences": {
    "tone": "formal_legal",
    "max_findings_per_doc": 25,
    "include_business_impact_summary": true
  }
}
```

In v1, this config is generated by the Builder agent during build mode and saved alongside the copilot ID.

9. API design

Single endpoint, two modes.

POST /run

Build mode request:

```
{
  "mode": "build",
  "intake": {
    "workflow_description": "I need to review vendor NDAs against our fintech's d
    "org_id": "fintech_co_001",
    "document_type": "nda",
    "risk_appetite": "conservative",
    "language": "en"
  }
}
```

language can be "en" or "ar" . The Interviewer agent processes both.

Build mode response:

```
{
  "run_id": "uuid",
  "mode": "build",
  "copilot_id": "cp_a7f3e2d1",
  "config": { /* full org adaptation JSON */ },
  "validation_results": { /* sample run summary */ },
  "audit_trail": [ /* full agent interaction log */ ]
}
```

Use mode request:

```
{
  "mode": "use",
  "copilot_id": "cp_a7f3e2d1",
  "document": {
    "type": "text",
    "content": "..."
  }
}
```

Use mode response:

```
{
  "run_id": "uuid",
  "mode": "use",
```

```
"copilot_id": "cp_a7f3e2d1",
"findings": [ /* array per clause */ ],
"summary": { /* counts, risk buckets, recommendations */ },
"audit_trail": [ /* full agent interaction log */ ]
}
```

10. The chat UI (port 8001)

Streamlit single-page app. The user's primary touchpoint and the main visual asset for the demo video.

Layout

- **Top:** mode toggle (Build / Use)
- **Center:** chat interface
- **Right sidebar:** audit trail (expandable per step), copilot list (in Use mode), org config preview (in Build mode)
- **Bottom:** input box with Arabic/English support

Build mode flow

1. User types description of need (English or Arabic). Submitted via `st.chat_input`.
2. UI calls `POST /run` with `mode: "build"`.
3. As the agent council runs, UI streams audit trail entries to the right sidebar in real-time.
4. Chat displays: Interviewer's follow-up questions, debate highlights, final config summary.
5. On completion, UI displays the `copilot_id` and a "Try it now" button that switches to Use mode pre-populated.

Use mode flow

1. User selects a `copilot_id` from a dropdown (populated from previously built copilots).
2. User uploads a document (text paste or file).
3. UI calls `POST /run` with `mode: "use"`.

4. Findings stream into the chat as the Reviewer + Citation Verifier loop runs.

5. Audit trail in sidebar shows every step.

Implementation requirements

- Streamlit ≥ 1.30
 - Uses `st.chat_message` and `st.chat_input` primitives
 - Calls the backend API at `http://localhost:8000/run` (configurable via env var)
 - Supports Arabic right-to-left rendering on input fields
 - No authentication in v1 (single-user demo)
 - Runs on port 8001 in its own process (`run_ui.py`)
-

11. Tech stack

Layer	Choice
Language	Python 3.11
Agent orchestration	LangGraph
API	FastAPI
UI	Streamlit
Vector store	ChromaDB (local, file-backed)
LLM provider	OpenAI-protocol compatible, configured via env vars
Embeddings	text-embedding-3-large
Container	Docker
Logging	Python logging → JSONL in <code>logs/</code>

LLM client implementation details: see SOW v2 section 6.

12. Repo strategy

Source of truth: GitLab. Developer commits all work to the founder’s GitLab repo from Day 1.

Mirror: GitHub. Maintained continuously, not just at submission. By May 30, the GitHub mirror must contain the complete codebase, documentation, and examples.

Mirror mechanism: GitLab’s built-in repository mirroring to a GitHub remote, configured Day 1.

Required GitHub contents by May 30:

- `app/` — full agent code
- `data/` — corpus and sample org configs
- `run.py` — API entry point (port 8000)
- `run_ui.py` — Streamlit UI entry point (port 8001)
- `requirements.txt`
- `Dockerfile`
- `metadata.json`
- `input_examples/` — at least 3 per mode (6 total), including Arabic input examples
- `output_examples/` — at least 3 per mode (6 total)
- `logs/` — sample logs showing all 5 feedback loops
- `.env.example`
- `README.md`
- `docs/architecture.md`

13. Build timeline

Today is May 21. Developer delivery deadline is May 30. Submission deadline is June 7.

Date	Day	Owner	Goal
Wed May 21	0	Founder + Claude	Share PRD v3 + SOW v2 with dev; receive quote; sign SOW

Thu May 22	1	Dev	Repo scaffold, GitLab→GitHub mirror, LLM client wrapper, LangGraph hello-world, Streamlit UI shell
Fri May 23	2	Dev	Corpus ingestion + embedding; Interviewer agent (incl. Arabic intake test); Debater A & B; Loop 1
Sat May 24	3	Dev	Architect + Builder + Validator; Loops 2 & 3; full build-mode end-to-end via API and UI
Sun May 25	4	Dev	Use-mode agents (Reviewer, Citation Verifier); Loop 4 (citation rejection)
Mon–Thu May 26–29	—	All	EID BREAK. No new code. Founder + Claude do: demo NDA drafting, README v1, sample org configs, Arabic test inputs
Fri May 30	5	Dev	Counter-Proposal Drafter + Synthesis; Loop 5; UI polished for both modes; all docs and examples
Sat May 31	6	Founder + Claude	Acceptance review; stress test
Sun Jun 1	7	Founder + Claude	Customer validation session with compliance/legal expert
Mon Jun 2	8	Founder + Claude	Incorporate validation feedback; input/output examples polish
Tue Jun 3	9	Founder + Claude	Demo video first cut
Wed Jun 4	10	Founder + Claude	Demo video polish + second validation session if available
Thu Jun 5	11	Founder + Claude	Submission compliance audit; metadata.json finalize
Fri Jun 6	12	All	Dry runs against unseen inputs; buffer
Sat Jun 7	—	Founder	Submit before 23:59 UAE time

14. Roles & responsibilities

Founder. Product direction, demo narrative, judge-facing storytelling, customer validation, final demo recording, submission execution.

Claude (me). CTO function: architecture decisions, agent prompt engineering, code review, daily check-ins with dev, troubleshooting, scope discipline, PRD revisions, submission compliance audit.

Hired developer. Build execution against PRD v3 and SOW v2. Daily updates. Blockers within 4 hours. Owns GitLab→GitHub mirror. Delivers full system by May 30.

15. Risks & mitigations

Risk	Likelihood	Impact	Mitigation
Jais not available on Compass	Medium	Medium	Fall back to GPT-4.1 with Arabic prompting; demo narrative updated; test Day 1
Developer doesn't hit May 30	Medium	High	Daily check-ins; scope-cut sequence pre-defined; trigger reviews May 24 and May 28
LLM quota errors during dev	Medium	Medium	<code>SAMPLE_MODE=true</code> throughout; aggressive embedding cache; integration test Day 1
Citation Verifier false rejects	Medium	High	Exact-text retrieval; tunable threshold; manual review of first 20 rejections
Multi-agent loops never converge	Low	High	Hard cap of 3 iterations per loop; human-review flag on escalation
Chat UI dev complexity underestimated	Medium	Medium	Streamlit chat primitives are mature; scope strictly limited to mode toggle + chat + sidebar; no custom CSS
Customer validation session falls through	Low	Medium	Two attempts planned (Jun 1 and Jun 4); founder has compliance + legal contacts

16. Demo script (2:45)

0:00 – 0:20 — The problem. A New York lawyer was sanctioned for filing a brief with hallucinated citations. Now imagine that problem at scale, in a jurisdiction where every organization reads the same statute differently, and where the authoritative legal text is in Arabic. Generic legal AI flattens that nuance. Wakeel doesn't.

0:20 – 0:50 — The factory in Arabic. Sarah types her workflow need in Arabic. The Interviewer understands. Debater A and B exchange arguments on-screen in English. The Architect synthesizes. The Builder mints a copilot in 60 seconds. The Validator runs sample inputs and approves.

0:50 – 1:30 — The copilot in action. Sarah uploads an NDA. The copilot — *her* copilot — finds three issues. Watch the Citation Verifier *reject* a hallucinated cite. Watch it come back with the correct article. Pause on the rejection log. This is the moment.

1:30 – 2:15 — The moat. Same NDA, same template, different org config. Fintech vs hospital. Findings differ. Risk levels differ. Counter-proposals differ. Two visibly different reviews of the same document because the interpretation layer is different. "Public law is public. Interpretation is private. We own the layer between."

2:15 – 2:45 — Architecture and vision. Quick diagram. Ten agents, two modes, five feedback loops, sovereign infrastructure, Arabic-aware. Vision: full bilingual coverage, free-zone laws, continuous learning. Roll credits.

17. Acceptance criteria for May 30 developer delivery

By 23:59 UAE on May 30, the GitHub repo must contain:

- Working `POST /run` on port 8000 supporting both `mode: "build"` and `mode: "use"`
- Streamlit UI running on port 8001 with mode toggle, chat interface, and audit trail
- All 10 agents implemented and wired via LangGraph
- All 5 feedback loops firing and logged
- Interviewer agent accepts and responds correctly to at least 3 Arabic test inputs
- NDA review copilot template working end-to-end
- Corpus ingested: UAE Federal Labour Law, PDPL, Commercial Transactions Law (key chapters), and Civil Transactions Law (contract articles)

- Shared LLM client wrapper (`app/llm.py`) per SOW section 6
 - 3+ input examples per mode (6 total, including ≥ 1 Arabic input)
 - 3+ corresponding output examples per mode
 - Dockerfile **builds**; `docker run -p 8000:8000 -p 8001:8001` **works from clean clone**
 - `README.md` **and** `docs/architecture.md` **complete**
 - `.env.example` **with no real secrets**
 - GitLab→GitHub mirror live and current
-

End of PRD v3.0.